NOMAD Shorts Maker 코드 리뷰

리뷰 일자: 2026-10-11 | 대상 경로: D:₩WorkSpace₩platform₩NOMAD Shorts Maker | src 파일 277개 / 약 26,300줄 (TS/TSX)

1. 프로젝트 개요

한 줄 설명

"NOMAD AI Factory (ShortsOS)" — 트렌드 레이더(MODULE 0)로 소재를 찾고, STT -> 번역 -> TTS(KO/JA/EN) -> 무음컷 -> 레이어 합성 -> 인코딩 -> Compliance 검증 -> 최대 40개 YouTube 채널 배포까지 자동화하는 멀티테넌트 크레딧 과금형 쇼츠 공장 SaaS. 타겟은 한국 1인 크리에이터, 이커머스 운영자, 콘텐츠 에이전시 (PRD.md:8-16).

단, 실제 코드베이스에는 이전 제품인 "NOMAD Content Automator"(네이버/티스토리/워드프레스/인스타그램 블로그 자동 포스팅 + 쿠팡파트너스)가 그대로 공존한다. package.json:2의 이름은 nomad-content-automator, 로그인 화면 브랜드는 "Content Automator"(src/app/login/page.tsx:59), 사이드바 브랜드는 "ShortsOS"(src/components/layout/Sidebar.tsx:44)로 두 제품이 섞여 있다.

기술 스택

영역	스택	비고
프론트	Next.js 16.2.9 (App Router), React 19.2, Tailwind v4, shadcn/base-ui, lucide, recharts, @xyflow/react(React Flow), TipTap	116개 파일이 'use client'
백엔드	Next.js Route Handlers 52개, Supabase Edge Functions 3개(Deno)	미들웨어/proxy 없음
DB/인증	Supabase (Postgres + RLS + Auth), @supabase/ssr	마이그레이션 3개(번호 충돌)
큐/워커	BullMQ + ioredis(Upstash), TS 워커 16개(src/lib/queue/workers), Python 워커 4개(workers/, Railway 가정)	두 종류 워커가 같은 큐 소비
AI	OpenAI(gpt-4o-mini, Whisper, TTS), OpenRouter(claude-sonnet-4-6), DeepL, ElevenLabs, Suno, fal	클라이언트 3종 혼재
미디어	FFmpeg, yt-dlp, Playwright(네이버/티스토리 포스팅, 브랜드커넥트)	
결제	Toss Payments SDK, 크레딧 RPC(deduct_credits/add_credits)	결제 경로 2개 공존
인프라	Vercel(cron * * * * * -> /api/schedules/run), Upstash Redis, Railway Python	

현재 완성도 추정: 약 35%

- UI 셸(13개 메뉴, 40여 페이지, 노드 29종)은 80% 이상 그려져 있으나, 대시보드 18개 페이지가 하드코딩 더미 데이터이고 실제 API/Supabase를 호출하는 페이지는 9개에 불과하다 (src/app/dashboard/page.tsx:12-50, src/app/dashboard/radar/page.tsx:6-48).
- API 라우트 52개 중 CRUD 성격(계정, 포스트, 스케줄, 쿠팡, 링크인바이오, 결제 기본)은 동작 가능한 수준이지만, 핵심인 숏폼 생산 파이프라인은 enqueue만 하고 의존 관계·워커 실행·결과 수집이 연결돼 있지 않다 (src/lib/queue/pipeline-track-a.ts:33-100).
- tsc --noEmit 결과 21개 타입 오류로 next build가 실패한다.
- 사용 중인 환경변수 24개(OPENAI_API_KEY, GOOGLE_CLIENT_ID, TOSS_SECRET_KEY, UPSTASH_REDIS_URL 등)가 .env.local에 없어 AI/OAuth/큐/결제 기능이 즉시 예외로 종료된다.

핵심 기능 목록

기능	상태
트렌드 레이더 (플레이보드/채널파인더/블링)	lib 구현, UI는 더미, 성장률은 난수
Track A/B/C 숏폼 파이프라인	enqueue 코드만 존재, 체인/워커 미연결
Track D 롱폼, Track E Lofi	UI + lib 골격
YouTube 40채널 OAuth/쿼터 관리	lib 구현, 콜백 인증 결함
크레딧 선차감/환불	RPC 구현, 권한 결함
Toss 결제/구독	결제 승인 흐름 구현, 웹훅/플랜 모델 불일치
블로그/인스타 멀티채널 포스팅(구 제품)	구현(Playwright, Meta Graph), 내부 호출 인증 우회
n8n/Make/ReactFlow 자동화	구현, 내부 API 호출 실패 버그
스케줄 큐(Vercel cron)	구현, RLS로 인해 cron 컨텍스트에서 무동작

2. 구조 분석

주요 디렉토리/파일과 역할

경로	역할	상태
src/app/page.tsx, src/app/pricing, src/app/login	마케팅/가격/로그인	579줄 단일 클라이언트 컴포넌트 랜딩, 가격은 lib/plans.ts 기준
src/app/dashboard/ * * (40 페이지)	대시보드 UI	인증 가드 없음, 대부분 더미 데이터
src/app/api/ * * (52 라우트)	백엔드	6개 라우트 인증 없음(웹훅/OAuth 콜백/추천)
src/app/landing/[slug], src/app/lb/[username]	공개 랜딩(ISR 1h), 링크인바이오	Next 16 params 미대응
src/lib/supabase/{client,server,types}.ts	Supabase 클라이언트	createServiceClient 정의만 있고 사용 0건
src/lib/billing/credits.ts, src/lib/plans.ts, src/themes/shorts-factory.ts	과금	플랜 정의 3종 상충
src/lib/queue/{bullmq,pipeline-track-a/b/c}.ts, workers/(각 .ts)	파이프라인/워커	워커 기동 스크립트 없음
src/lib/{ai,media,platforms,radar,channels,compliance,landing}	외부 연동/도메인 로직	대체로 얇은 fetch 래퍼
src/skill/ * *	구 제품 엔진(블로그 AI 생성, Playwright 포스팅, n8n/Make 브릿지, 스케줄 큐)	동작 가능하나 신제품과 미통합
src/taste/content-automator/ * vs src/themes/shorts-factory.ts	테마/설정	두 제품의 테마가 각각 존재
src/components/ * * (약 100개)	UI 컴포넌트	MultiChannelEditor.tsx 517줄 등 대형 컴포넌트
supabase/migrations/001_initial.sql	구 제품 스키마(user_id 기반, 9 테이블)	번호 001 중복
supabase/migrations/001_initial_schema.sql	신제품 스키마(org 기반, 17 테이블, 크레딧 RPC)	번호 001 중복
supabase/migrations/002_billing.sql	구독/결제 이력(user_id 기반)	org 크레딧과 별개
supabase/functions/ *	Edge Functions(서비스 롤 사용, 서명 검증)	Next 라우트와 기능 중복
workers/ * .py	Railway Python 워커(Whisper/FFmpeg/무음컷/합성)	TS 워커와 큐 중복, 페이로드 계약 불일치

경로	역할	상태
루트 .md(루트 전체), taste-skill-shorts-factory 계열 .ts, files.zip	스펙 문서 14개 + 동일 내용 zip	중복/버전 분기

데이터 흐름 요약

클라이언트(모두 'use client')가 /api/* 를 fetch하고, 라우트는 createClient()(쿠키 기반 anon 키)로 Supabase에 접근해 RLS에 의존한다. 숏폼 작업은 POST /api/jobs가 jobs 레코드를 만든 뒤 runTrackA/B/C를 await 없이 호출하고, 파이프라인은 BullMQ 큐 7~8개에 모든 단계를 한 번에 넣고 곧바로 status='done'으로 갱신한다. 워커는 src/lib/queue/workers/*.ts에 정의되어 있으나 어디서도 기동되지 않으며, 내부에서 next/headers의 cookies()를 쓰는 클라이언트를 사용하므로 Next 요청 컨텍스트 밖에서는 실행이 불가능하다. 결제는 Toss 승인(/api/billing/confirm) -> subscriptions 테이블(user 기반)로 흐르고, 별도로 /api/webhooks/toss가 organizations.credit_balance(org 기반)를 갱신하는 두 번째 경로가 있어 서로 연결되지 않는다. 스케줄 큐는 Vercel cron이 매분 /api/schedules/run을 호출하지만 세션이 없는 anon 클라이언트로 조회하므로 RLS에 걸려 항상 0건을 처리한다.

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	보안	supabase/migrations/001_initial_schema.sql:455-499	deduct_credits/add_credits가 SECURITY DEFINER인데 authenticated 역할의 EXECUTE가 회수되지 않음. 로그인한 누구나 브라우저에서 supabase.rpc('add_credits', {p_org_id, p_amount: 999999}) 호출 가능
2	높음	보안	src/app/api/webhooks/toss/route.ts:19-28, 38-51	서명 헤더가 없으면 검증을 건너뜀(if (signature)). 임의 orderId="{orgId}_{planId}_x" POST로 플랜 변경 + 크레딧 충전 가능
3	높음	버그	src/app/api/billing/checkout/route.ts:18 vs src/app/api/webhooks/toss/route.ts:37-38	checkout은 nomad-{uuid} 형식, 웹훅은 {orgId}_{planId}_{ts}를 기대 -> 정상 결제는 웹훅에서 처리되지 않음
4	높음	보안	src/app/api/posts/[id]/publish/route.ts:23-30, src/app/api/schedules/run/route.ts:19-25	x-internal-call 헤더 하나로 인증 우회. 외부에서 헤더만 붙이면 타인 포스트 발행/큐 실행 트리거 가능
5	높음	보안	src/app/api/channels/oauth/route.ts:16-18	OAuth 콜백에 로그인 검증 없음. state(base64 JSON)의 orgId를 그대로 신뢰 -> 공격자가 자기 Google 채널을 타인 org에 연결. CSRF용 state 검증도 없음
6	높음	보안	src/app/dashboard/layout.tsx:4-14, src/lib/mock-auth.ts:15-19, src/app/login/page.tsx:17-20, 77-83	미들웨어/proxy 없음, 대시보드 레이아웃에 세션 확인 없음. Mock 로그인(localStorage)은 Supabase 설정 여부와 무관하게 항상 노출
7	높음	보안	src/lib/crypto.ts:4	ENCRYPTION_KEY 미설정 시 '0'.repeat(64) 키로 폴백 -> OAuth 토큰/계정 비밀번호가 사실상 공개 키로 암호화됨
8	높음	버그	src/lib/supabase/server.ts:4-25, 27-48; 사용처 src/app/api/webhooks/toss/route.ts:35, src/app/api/schedules/run/route.ts:27,58, src/lib/queue/workers/stt.worker.ts:18, src/lib/queue/pipeline-track-a.ts:22	웹훅/cron/워커처럼 세션이 없는 컨텍스트에서 쿠키 기반 anon 클라이언트 사용 -> RLS로 모든 쓰기가 0행 처리되고 에러도 확인하지 않음. createServiceClient는 정의만 있고 호출 0건

#	심각도	분류	위치(경로:줄)	내용
9	높음	버그	src/lib/queue/pipeline-track-a.ts:33-100, src/app/api/jobs/route.ts:102-116	단계 간 의존 없이 8개 큐에 동시 enqueue, '__from_stt__' 같은 placeholder 전달, enqueue 직후 status='done'. 라우트에서 fire-and-forget 호출이라 Vercel에서는 응답 후 프로세스가 종료됨
10	높음	구조	src/lib/queue/workers/stt.worker.ts:6,18, package.json:5-10, workers/whisper_worker.py:38,60-63	TS 워커가 next/headers에 의존해 독립 실행 불가, 기동 스크립트 없음. Python 워커가 같은 큐(nomad:stt)를 소비하되 입력 계약이 다름(videoPath vs audioUrl)
11	높음	설정	src/lib/queue/bullmq.ts:86, src/lib/queue/workers/publish.worker.ts:40,47, translate.worker.ts:41, suno.worker.ts:26-38, src/lib/radar/playboard.ts:44-86, channel-finder.ts:47, src/lib/platforms/threads.ts:249, instagram.ts:243	tsc 21개 오류 -> next build 실패. ioredis 중복 설치(루트 vs bullmq 내부), 반환 타입 불일치(uploadVideo가 string 반환인데 .videoId 접근), 키 대소문자 불일치(KO vs ko)
12	높음	버그	src/app/landing/[slug]/page.tsx:7,10-17,24,38,46, src/app/api/jobs/[id]/route.ts:6,22,33	Next 16에서 params는 Promise인데 동기 객체로 사용. generateStaticParams에서 cookies() 기반 클라이언트 호출(빌드 시 실패). increment_landing_views RPC가 마이그레이션에 없음
13	중간	구조	src/lib/plans.ts:1,18-71, src/themes/shorts-factory.ts:61-65, CLAUDE_pricing.md:10-25, src/app/api/webhooks/toss/route.ts:8-13	플랜 모델 3종 상충(free/pro/business, free/starter/pro/agency, starter/basic/pro/advanced). jobs/route.ts:61과 oauth-manager.ts:49는 starter/agency 기준, 결제 UI는 pro/business 기준
14	중간	구조	supabase/migrations/001_initial.sql, 001_initial_schema.sql, 002_billing.sql	마이그레이션 번호 001 2개. user_id 기반 스키마와 org 기반 스키마, subscriptions와 organizations.credit_balance 과금이 병존. 어느 쪽이 정본인지 코드로 판단 불가
15	중간	버그	src/skill/automation-engine/reactflow-executor.ts:90-94 vs src/app/api/ai/generate/route.ts:7-10	내부 호출이 x-internal-call을 보내지만 대상 라우트는 getUser를 요구해 항상 401. 바디도 {title,tags}를 보내는데 라우트는 keyword 필수 -> AIContentGen 노드는 조용히 무동작
16	중간	보안	src/app/api/webhooks/distrokid/route.ts:4-24, src/app/api/automation/webhook/callback/route.ts:52-72	DistroKid 웹훅은 인증 전무. 자동화 콜백은 서명이 있을 때만 검증하고 automation_configs를 user_id 없이 mode로만 조회
17	중간	품질	src/app/dashboard/page.tsx:12-50, src/app/dashboard/radar/page.tsx:6-48, src/lib/radar/playboard.ts:75-76	18개 대시보드 페이지가 하드코딩 더미. 플레이보드 스크래퍼는 growthRate = Math.random() * 3+0.5를 실제 지표로 DB에 저장
18	중간	성능	src/lib/platforms/youtube.ts:54-62	readFileSync로 영상 전체를 메모리에 올려 단일 PUT. 수백 MB 영상이면 서버리스 메모리 초과
19	중간	성능	src/lib/queue/bullmq.ts:4-8	모듈 로드 시점에 Redis 연결 생성. jobs/route.ts가 import하므로 모든 콜드스타트에서 연결 시도, 미설정 시 localhost:6379
20	중간	설정	.env.local, src/lib/compliance/filter.ts:3, src/lib/ai/whisper.ts:22, src/lib/channels/oauth-manager.ts:5-7	코드가 참조하는 env 24개가 .env.local/.env.local.example에 없음(OPENAI_API_KEY, GOOGLE_CLIENT_, TOSS_, UPSTASH_REDIS_URL, ELEVENLABS_*, SERPAPI_KEY 등)
21	중간	구조	src/skill/ai-engine/content-generator.ts:3-8, src/lib/compliance/filter.ts:3, src/lib/ai/openai-chat.ts:16-28	LLM 호출 방식 3종(OpenRouter SDK, OpenAI SDK, raw fetch) 혼재. 모델/키/에러 처리 정책이 제각각

#	심각도	분류	위치(경로:줄)	내용
22	중간	품질	src/app/api/jobs/route.ts:22, 53-57, src/app/api/accounts/route.ts:25-31, src/app/api/billing/confirm/route.ts:12	입력 검증 부재: limit 무제한, track enum 미검증, credentials 임의 객체 암호화 저장. zod 등 스키마 검증 미사용
23	중간	버그	src/app/pricing/page.tsx:39	Toss customerKey를 user-${Date.now()}로 생성 -> 사용자와 매핑 불가, 빌링키 정기결제 불가
24	중간	버그	src/app/api/jobs/route.ts:12-17	org_members를 .single()로 조회 -> 사용자가 2개 조직에 속하면 전체 API가 에러
25	낮음	구조	src/app/dashboard/page.tsx:1, 116개 파일	페이지 전부 클라이언트 컴포넌트. 서버 컴포넌트 데이터 페칭·스트리밍 미활용
26	낮음	품질	루트 CLAUDE_add.md, CLAUDE_pricing.md, DESIGN_add.md, DESIGN_complete.md, DESIGN_pricing.md, PRD_add.md, PRD_pricing.md, taste-skill-shorts-factory * .ts 3종, files.zip, README.md	스펙 문서가 본문/추가분/가격분으로 분기되어 14개, files.zip은 동일 파일 압축본. README는 create-next-app 기본
27	낮음	품질	src/lib/queue/workers/stt.worker.ts:54-55 외 17건	console.log 직접 사용, 이모지 로그. 구조화 로거 없음
28	낮음	버그	src/app/api/webhooks/toss/route.ts:30	JSON.parse(rawBody) 예외 미처리 -> 잘못된 바디에 500
29	낮음	보안	src/components/posts/MultiChannelEditor.tsx:316	AI 생성 콘텐츠를 dangerouslySetInnerHTML로 렌더. sanitize 없음

심각도 높음 항목 상세

#1 크레딧 RPC 권한. add_credits는 security definer로 RLS를 우회해 organizations.credit_balance를 증가시키는데, Supabase 기본값으로 public 스키마 함수는 anon/authenticated가 실행할 수 있다. 호출부(src/lib/billing/credits.ts:77-92)는 서버 전용이지만, 같은 함수가 브라우저 anon 키로도 호출 가능하므로 로그인 사용자는 임의 금액을 자기 조직에 충전할 수 있다. 과금 모델의 근간이 무너지는 결함이며 revoke execute ... from anon, authenticated 후 서비스 롤 전용으로 돌리거나, 함수 내부에서 auth.uid() 기반 멤버십·호출 맥락을 검증해야 한다.

#2/#3 Toss 웹훅. 서명 검증이 "헤더가 있을 때만" 수행되므로 헤더를 생략하면 그대로 통과한다. 이어서 orderId.split('_')로 orgId와 planId를 꺼내 플랜을 바꾸고 크레딧을 충전한다(toss/route.ts:38-51). 실제 checkout은 nomad-{uuid}를 발급하므로 정상 결제는 이 분기에 들어오지 않고, 오히려 공격자가 조작한 요청만 처리된다. 또한 이 라우트는 세션 없는 anon 클라이언트를 쓰므로 현재는 RLS 덕분에 실제 반영이 막혀 있을 가능성이 높지만, #8을 고치는 순간 취약점이 활성화된다.

#4 x-internal-call 우회. 스케줄 큐가 자기 자신의 /api/posts/:id/publish를 HTTP로 호출하기 위해 만든 편의 헤더가 공개 인터넷에서 그대로 통한다. 발행 라우트는 헤더만 있으면 blog_posts.user_id를 읽어 그 사용자 권한으로 처리하고, 큐 실행 라우트는 CRON_SECRET 비교 자체를 건너뛴다. 내부 호출은 HTTP 왕복이 아니라 함수 직접 호출로 바꾸고, 불가피하면 CRON_SECRET과 동등한 비밀값 비교를 강제해야 한다.

#5 YouTube OAuth 콜백. Google이 리다이렉트하는 콜백은 로그인 세션을 확인하지 않고 state를 디코딩해 orgId를 꺼낸다. state는 클라이언트가 만든 base64 JSON이므로 서명이 없고, 결국 임의 org에 채널을 삽입할 수 있다. 콜백에서 getUser()로 세션을 확인하고, state는 서버에서 발급한 난수를 세션/DB에 저장해 대조하는 표준 CSRF 패턴으로 바꿔야 한다.

#6 대시보드 무인증. Next 16용 proxy.ts(구 middleware)가 없고 dashboard/layout.tsx는 세션을 보지 않는다. 각 페이지가 더미 데이터라 현재 노출되는 정보는 적지만, 실데이터 연결 즉시 비로그인 접근이 가능해진다. 또 Mock 로그인은 isSupabaseConfigured()와 무관하게 버튼이 항상 렌더되어 프로덕션 빌드에 남는다.

#7 암호화 키 폴백. 모든 OAuth 토큰과 네이버/티스토리 계정 비밀번호가 ENCRYPTION_KEY로 AES-256-GCM 암호화되는데, 미설정 시 전부 0인 키를 쓴다. 배포 환경에서 변수 누락이 있어도 예외 없이 동작하므로 사실상 평문 저장과 같다. 미설정이면 기동 시점에 throw 해야 한다.

#8 서비스 클라이언트 미사용. 웹훅·cron·워커는 요청 쿠키가 없으므로 createClient()는 익명 세션이고 RLS는 모든 행을 숨긴다. 코드가 error를 확인하지 않아 "성공처럼 보이지만 아무 일도 안 일어나는" 상태다. Vercel cron이 매분 돌아도 schedules는 영원히 pending이다. 백그라운드 컨텍스트 전용으로 SUPABASE_SERVICE_ROLE_KEY를 쓰는 순수 createClient(url, serviceKey)(쿠키 비의존)를 분리하고, 모든 쓰기의 error를 처리해야 한다.

#9/#10 파이프라인과 워커. BullMQ에는 부모-자식 의존을 표현하는 FlowProducer가 있지만 현재는 독립 addJob 8회로 끝나며, STT 결과를 다음 단계에 전달할 방법이 없어 placeholder 문자열을 넘긴다. 워커 파일은 import 즉시 new Worker()를 만들지만 실행 진입점이 없고, 내부에서 cookies()를 쓰는 클라이언트 때문에 Node 단독 실행도 불가능하다. Python 워커는 동일 큐 이름을 소비하지만 BullMQ의 Lua 기반 프로토콜을 Redis 명령으로 흉내 내는 구조여서 TS 워커와 경쟁·충돌한다. 어느 런타임이 워커인지 결정하고 하나만 남겨야 한다.

#11/#12 빌드 불가. 21개 타입 오류와 Next 16 params Promise 미대응은 즉시 배포를 막는다. generateStaticParams는 빌드 시점에 실행되므로 cookies()를 호출하는 클라이언트를 쓸 수 없다.

4. 효율성 개선 제안

구조/아키텍처

- 제품 경계를 먼저 확정한다. "Content Automator"(블로그 포스팅, src/skill/ * * , 001_initial.sql, src/taste/content-automator)와 "AI Factory"(src/lib/ * * , 001_initial_schema.sql, src/themes/shorts-factory.ts)를 한 저장소에 두더라도 스키마·플랜·테마는 하나로 통합하거나, 구 제품을 별도 패키지로 격리해야 한다. 지금은 jobs API가 starter/agency 플랜을 보고 결제 UI는 pro/business를 팔고 있다.
- Supabase 접근 계층을 3종으로 명시한다: 브라우저(anon+세션), 서버 요청(쿠키 SSR), 백그라운드(service role, 쿠키 비의존). 웹훅/cron/워커는 반드시 세 번째를 쓰고 org 소유권은 코드에서 검증한다.
- 파이프라인은 BullMQ FlowProducer로 부모(job) 아래 단계 자식을 체인하고, 각 워커는 이전 단계의 returnvalue를 읽는다. 워커는 workers/ 패키지로 분리해 tsx/node로 독립 기동하고 Next 의존(next/headers)을 제거한다. Python 워커는 FFmpeg/Whisper 전용으로 남기되 TS 워커와 큐를 나눈다.
- 내부 자기 호출(fetch(APP_URL + '/api/...') + x-internal-call)을 모두 함수 직접 호출로 치환한다(queue-processor.ts:63, reactflow-executor.ts:90).

성능

- YouTube 업로드는 파일 스트림(fs.createReadStream) 또는 resumable chunk 업로드로 바꾼다.
- Redis 연결은 lazy 싱글턴으로 바꾸고 API 라우트에서 BullMQ를 import하지 않도록 enqueue 전용 얇은 모듈을 둔다.
- 대시보드 페이지를 서버 컴포넌트로 전환해 초기 데이터는 서버에서 조회하고, 인터랙션 부분만 클라이언트 컴포넌트로 분리한다. jobs 목록처럼 자주 바뀌는 데이터는 SWR/React Query로 폴링한다.
- org_members 조회가 거의 모든 라우트에서 반복된다. getCurrentOrg(supabase) 헬퍼로 묶고 요청 단위 캐시를 둔다.

개발 생산성

- tsc --noEmit과 eslint를 CI 게이트로 둔다. ioredis는 bullmq가 요구하는 버전으로 맞추고 루트 직접 의존을 제거한다(package.json:21).
- 모든 라우트 입력에 zod 스키마를 두고, process.env는 기동 시 zod로 검증하는 lib/env.ts로 단일화한다(현재 누락 24개).
- LLM 클라이언트를 lib/ai/client.ts 하나로 모아 모델/키/재시도/JSON 파싱 정책을 공유한다(claude-api 참고: OpenRouter 경유 Claude와 OpenAI 직접 호출을 한 인터페이스로).

- 플랜 상수는 src/lib/plans.ts 단일 소스로 두고 DB plans 테이블과 동기화한다. themes/shorts-factory.ts의 plans와 webhooks/toss의 PLAN_CREDITS는 제거한다.
- 테스트가 0개다. 최소한 크레딧 RPC 권한, 웹훅 서명, OAuth state, RLS 격리에 대한 통합 테스트를 Supabase 로컬 스택으로 둔다.

운영/배포

- .env.local.example을 코드가 참조하는 전 키로 갱신하고, 필수 키 누락 시 빌드/기동을 실패시킨다.
- Vercel cron `* * * * *`는 Hobby 플랜에서 허용되지 않으므로 플랜 확인 또는 Supabase pg_cron -> Edge Function(process-schedule) 경로 하나로 통일한다. 현재 Next 라우트와 Edge Function이 같은 기능을 중복 구현한다.
- 루트 문서를 docs/ 아래 PRD.md, DESIGN.md, CLAUDE.md 3개로 병합하고 _add/_pricing/_complete/taste-skill 계열 .ts/files.zip은 삭제한다. src/themes/shorts-factory.ts가 이미 taste 파일의 실제 소스다.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	크레딧 RPC 봉쇄: add_credits/deduct_credits에 revoke execute from anon, authenticated, 서비스 롤 전용 호출로 전환, 함수 내부 org 멤버십 검증 추가, 회귀 테스트	P0	S	Fable 5.1	돈이 직접 걸린 권한 로직, 실수 비용 최대
2	결제 경로 단일화: Toss confirm 성공 시 플랜·크레딧 지급을 서버에서 처리, 웹훅은 서명 필수 + payment_history.order_id 매핑으로만 처리, PLAN_CREDITS/orderId 파싱 제거, customerKey를 user.id 기반으로	P0	M	Fable 5.1	결제/과금 모델 재설계, 두 스키마 간 정합성 필요
3	백그라운드 전용 Supabase 서비스 클라이언트(쿠키 비의존) 도입 후 웹훅·cron·워커·파이프라인에 적용, 모든 쓰기 error 처리, x-internal-call 우회 제거 및 내부 호출을 함수 직접 호출로 전환	P0	M	Fable 5.1	인증 우회 제거 + 다중 모듈 횡단 변경
4	인증 가드: src/proxy.ts에서 /dashboard/* *, /api/* *(웹훅 제외) 세션 검사, Mock 로그인을 NODE_ENV!=='production' && !isSupabaseConfigured()일 때만 렌더, YouTube/Meta OAuth state를 서버 발급 난수+세션 대조로 교체, ENCRYPTION_KEY 미설정 시 throw	P0	M	Fable 5.1	인증/권한/암호화 보안 로직
5	빌드 복구: tsc 21개 오류 수정, params Promise 대응(landing/[slug], jobs/[id]), generateStaticParams를 서비스 클라이언트로, ioredis 중복 제거, increment_landing_views 마이그레이션 추가	P0	S	Sonnet 5.5	명확한 타입/린트 오류 정리
6	제품 경계 확정 및 스키마 통합: 마이그레이션 번호 재정렬, user 기반 <-> org 기반 중 하나로 통일(권장: org), subscriptions를 org에 연결, 플랜 정의를 lib/plans.ts 단일 소스로, src/taste/content-automator 정리	P1	L	Fable 5.1	데이터 모델 재설계, 전 모듈 영향
7	파이프라인 재설계: BullMQ FlowProducer로 단계 체인, 워커를 workers/ 독립 패키지(Node 진입점 + npm run worker)로 분리, next/headers 의존 제거, Python 워커와 큐 분리 및 페이로드 계약 문서화, job 상태를 워커 완료 이벤트로 갱신	P1	L	Fable 5.1	아키텍처 재설계, 원인 복합적
8	전 API 라우트 zod 입력 검증(track enum, limit 상한, credentials 스키마, 결제 바디), getCurrentOrg 헬퍼로 org_members 조회 통합(복수 조직 대응)	P1	M	Opus 5.5	명확히 정의된 중간 규모 리팩토링

#	작업	우선순위	예상 규모	권장 모델	선정 이유
9	대시보드 실데이터 연결: dashboard/radar/jobs/channels/credits 페이지를 서버 컴포넌트 + 실제 API로 전환, 더미 배열 제거, playboard.ts 난수 성장률을 실제 계산 또는 명시적 is_mock 플래그로	P1	L	Opus 5.5	다수 파일 API 연동, 상태관리 정리
10	AI 클라이언트 단일화(lib/ai/client.ts) + lib/env.ts zod 환경변수 검증 + .env.local.example 전 키 반영	P1	S	Opus 5.5	API 연동 통합, 중간 난이도
11	reactflow-executor AIContentGen 내부 호출을 generateBlogContent 직접 호출로 교체, 바디 계약(keyword) 일치, DistroKid/자동화 콜백 웹훅 서명 필수화	P1	S	Opus 5.5	범위가 명확한 버그 수정
12	YouTube 업로드 스트리밍(createReadStream 또는 chunked resumable), Redis lazy 싱글턴 및 enqueue 전용 모듈 분리	P2	S	Opus 5.5	성능 개선, 2~3개 파일
13	루트 문서 정리: _add/_pricing 접미 문서/DESIGN_complete 내용을 PRD.md/DESIGN.md/CLAUDE.md에 병합, taste-skill 계열 .ts 3종·files.zip 삭제, README 실제 내용 작성, CLAUDE.md 파일 구조 섹션을 실제 src/ 구조로 갱신	P2	S	Sonnet 5.5	문서화/정리 작업
14	로깅 정리: console.log 17건을 공용 logger로, 이모지 제거, as never 2건 타입 보강, Toss 웹훅 JSON.parse 예외 처리, dangerouslySetInnerHTML sanitize	P2	S	Sonnet 5.5	단순 품질 정리
15	크레딧 RPC·웹훅 서명·OAuth state·RLS 격리에 대한 Supabase 로컬 통합 테스트 설계 및 CI(tsc/eslint/test) 구성	P2	M	Opus 5.5	테스트 설계, 중간 규모

6. 종합 평가

강점

1. 도메인 범위가 명확히 문서화되어 있다. PRD/DESIGN/CLAUDE.md가 5-Track, MODULE 0, 크레딧 비용표, 레이어 합성 순서까지 수치로 정의해 구현 기준이 분명하다.
2. 보안 의도는 코드에 드러난다. RLS 전 테이블 활성화, AES-256-GCM 토큰 암호화, HMAC 콜백 검증, Toss 서버측 승인, 크레딧 원자적 차감(for update) 등 올바른 패턴을 시도했다.
3. UI 완성도와 일관성이 높다. 디자인 토큰(색상/트랙/언어 배지)이 일관되게 적용되고 shadcn 기반 컴포넌트 구성이 깔끔하다. React Flow 노드 29종 등 제품 시연에 필요한 화면은 거의 갖춰졌다.

약점

1. 두 제품이 한 코드베이스에 섞여 스키마·플랜·테마·결제 경로가 각각 2~3개씩 존재하고 서로를 참조하지 않는다.
2. 보안 패턴을 시도했지만 마지막 한 단계가 빠져 있다(RPC 권한 미회수, 서명 "있을 때만" 검증, 헤더 하나로 우회, state 미검증, 키 폴백). 과금·인증 핵심이 모두 열려 있다.
3. 비동기 파이프라인이 실제로는 실행되지 않는다. 워커 기동 경로가 없고, 세션 없는 컨텍스트가 RLS에 막혀 조용히 실패하며, 빌드조차 통과하지 않는다.

모델별 작업 비율

- Fable 5.1: 6건 (40%) — P0 보안/결제 4건 + P1 아키텍처 2건
- Opus 5.5: 6건 (40%) — 입력 검증, 실데이터 연결, AI 클라이언트, 버그 수정, 성능, 테스트
- Sonnet 5.5: 3건 (20%) — 빌드 복구, 문서 정리, 로깅 정리

권장 진행 순서

1. 1단계 (P0, 1~2주): 작업 5(빌드 복구)로 배포 가능 상태를 만든 뒤 작업 1->3->2->4 순으로 과금·인증 구멍을 닫는다. 이 단계가 끝나기 전에는 실사용자 유입을 막아야 한다.

2. 2단계 (P1, 3~5주): 작업 6(제품 경계/스키마 통합)을 먼저 결정하고, 그 위에서 작업 7(파이프라인 재설계)과 작업 8/10/11을 진행한다. 작업 9(실데이터 연결)는 7이 어느 정도 동작해야 의미가 있으므로 마지막에 둔다.
3. 3단계 (P2, 1~2주): 작업 12~15로 성능·문서·로깅·테스트를 정리하고 CI 게이트를 고정한다.

주 담당 모델: Fable 5.1

이 프로젝트의 병목은 코드량이 아니라 "결정"이다. 어느 제품을 남길지, user 기반과 org 기반 중 어느 스키마가 정본인지, 결제는 구독형인지 크레딧형인지, 워커는 TS인지 Python인지가 정해지지 않은 채 양쪽 코드가 쌓였다. 또 남아 있는 결함 대부분이 인증·권한·결제·암호화·비동기 파이프라인처럼 실수 비용이 큰 영역이고, 원인이 여러 모듈에 걸쳐 있다(RLS × 세션 없는 컨텍스트 × 에러 미확인 × 내부 HTTP 호출). 따라서 1~2단계의 설계 결정과 보안·파이프라인 재구성은 Fable 5.1이 주도하고, 결정이 내려진 뒤의 API 연동·화면 연결·검증 추가는 Opus 5.5가, 빌드 오류·문서·로깅 정리는 Sonnet 5.5가 맡는 구성이 비용 대비 가장 안전하다.